

```

/* =====
PROCEDURES IN ORACLE PL/SQL

Topics:
- What is Procedure
- Procedure Syntax
- IN Parameter
- OUT Parameter
- IN OUT Parameter
- Local Variables
- Calling Procedures

Labs:
- Employee Bonus Procedure
- Telecom Recharge Procedure
- Bank Transfer Procedure
===== */

SET SERVEROUTPUT ON SIZE UNLIMITED;

/* =====
1. DROP OLD TABLES
===== */

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE employees';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE telecom_accounts';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE bank_accounts';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

/* =====
2. CREATE TABLES
===== */

CREATE TABLE employees
(
    employee_id    NUMBER PRIMARY KEY,
    employee_name  VARCHAR2(100),
    department     VARCHAR2(50),

```

```

        salary      NUMBER,
        bonus        NUMBER,
        status        VARCHAR2(20)
    );

```

```

CREATE TABLE telecom_accounts
(
    subscriber_id    NUMBER PRIMARY KEY,
    msisdn            VARCHAR2(20),
    customer_name     VARCHAR2(100),
    balance           NUMBER,
    status            VARCHAR2(20)
);

```

```

CREATE TABLE bank_accounts
(
    account_id        NUMBER PRIMARY KEY,
    account_name       VARCHAR2(100),
    balance            NUMBER,
    status             VARCHAR2(20)
);

```

```

/* =====
3. INSERT SAMPLE DATA
===== */

```

```

INSERT INTO employees VALUES (1, 'Aung Aung', 'IT', 900000, 0, 'ACTIVE');
INSERT INTO employees VALUES (2, 'Mg Mg', 'HR', 600000, 0, 'ACTIVE');
INSERT INTO employees VALUES (3, 'Su Su', 'Finance', 1200000, 0,
'ACTIVE');

```

```

INSERT INTO telecom_accounts VALUES (1, '091111111', 'Aung Aung', 5000,
'ACTIVE');
INSERT INTO telecom_accounts VALUES (2, '092222222', 'Mg Mg', 1000,
'ACTIVE');

```

```

INSERT INTO bank_accounts VALUES (1, 'Aung Aung', 1000000, 'ACTIVE');
INSERT INTO bank_accounts VALUES (2, 'Mg Mg', 500000, 'ACTIVE');

```

```

COMMIT;

```

```

/* =====
4. WHAT IS PROCEDURE?

```

```

A procedure is a named PL/SQL block.
It is stored in the database.
It can perform actions such as:
- INSERT
- UPDATE
- DELETE
- Business logic processing

```

```

Basic Syntax:

```

```

CREATE OR REPLACE PROCEDURE procedure_name
(
    parameter_name IN data_type
)
AS
    local_variable data_type;
BEGIN
    statements;
EXCEPTION
    exception_handling;
END;
/
===== */

/* =====
5. SIMPLE PROCEDURE WITHOUT PARAMETER
===== */

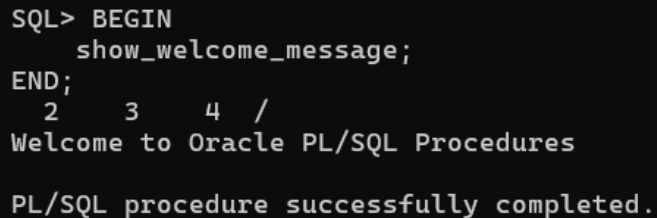
```

```

CREATE OR REPLACE PROCEDURE show_welcome_message
AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Welcome to Oracle PL/SQL Procedures');
END;
/

/* Calling Procedure */

```



```

SQL> BEGIN
      show_welcome_message;
END;
2      3      4  /
Welcome to Oracle PL/SQL Procedures

PL/SQL procedure successfully completed.

```

```

/* =====
6. PROCEDURE WITH IN PARAMETER

IN parameter is used to pass value into procedure.
Default parameter mode is IN.
===== */

```

```

CREATE OR REPLACE PROCEDURE show_employee
(
    p_employee_id IN employees.employee_id%TYPE
)
AS
    v_employee_name employees.employee_name%TYPE;
    v_department     employees.department%TYPE;
    v_salary          employees.salary%TYPE;
BEGIN
    SELECT employee_name, department, salary
    INTO v_employee_name, v_department, v_salary

```

```

FROM employees
WHERE employee_id = p_employee_id;

DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_employee_name);
DBMS_OUTPUT.PUT_LINE('Department: ' || v_department);
DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found.');
```

END;
/

/* Calling Procedure with IN Parameter */

```

SQL> BEGIN
      show_employee(3);
END;
/
  2      3      4  Employee Name: Su Su
Department: Finance
Salary: 1200000

PL/SQL procedure successfully completed.
```

/* =====
7. PROCEDURE WITH OUT PARAMETER

OUT parameter returns value from procedure.
===== */

```

CREATE OR REPLACE PROCEDURE get_employee_salary
(
    p_employee_id IN employees.employee_id%TYPE,
    p_salary      OUT employees.salary%TYPE
)
AS
BEGIN
    SELECT salary
    INTO p_salary
    FROM employees
    WHERE employee_id = p_employee_id;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        p_salary := 0;

END;
/
```

```
/* Calling Procedure with OUT Parameter */
```

```
SQL> DECLARE
      v_salary employees.salary%TYPE;
BEGIN
      get_employee_salary(3, v_salary);

      DBMS_OUTPUT.PUT_LINE('Returned Salary: ' || v_salary);
END;
/
 2      3      4      5      6      7      8 Returned Salary: 1200000
PL/SQL procedure successfully completed.
```

```
/* =====
8. PROCEDURE WITH IN OUT PARAMETER

IN OUT parameter receives value and returns modified value.
===== */
```

```
CREATE OR REPLACE PROCEDURE increase_amount
(
      p_amount IN OUT NUMBER
)
AS
BEGIN
      p_amount := p_amount + 10000;
END;
/
```

```
/* Calling Procedure with IN OUT Parameter */
```

```
SQL> DECLARE
      v_amount NUMBER := 50000;
BEGIN
      DBMS_OUTPUT.PUT_LINE('Before Procedure: ' || v_amount);

      increase_amount(v_amount);

      DBMS_OUTPUT.PUT_LINE('After Procedure: ' || v_amount);
END;
/
 2      3      4      5      6      7      8      9     10 Before Procedure: 50000
After Procedure: 60000
PL/SQL procedure successfully completed.
```

```

/* =====
9. LOCAL VARIABLES IN PROCEDURE

Local variables are declared in AS/IS section.
They can be used only inside the procedure.
===== */

CREATE OR REPLACE PROCEDURE calculate_monthly_tax
(
    p_employee_id IN employees.employee_id%TYPE
)
AS
    v_salary employees.salary%TYPE;
    v_tax     NUMBER;
BEGIN
    SELECT salary
    INTO v_salary
    FROM employees
    WHERE employee_id = p_employee_id;

    v_tax := v_salary * 0.05;

    DBMS_OUTPUT.PUT_LINE('Monthly Salary: ' || v_salary);
    DBMS_OUTPUT.PUT_LINE('Monthly Tax 5%: ' || v_tax);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found.');
```

```

END;
/

/* Calling Procedure */

```

```

SQL> BEGIN
      calculate_monthly_tax(2);
END;
/
  2      3      4 Monthly Salary: 600000
Monthly Tax 5%: 30000

PL/SQL procedure successfully completed.

```

```

/* =====
LAB 1: EMPLOYEE BONUS PROCEDURE

Requirement:
- Calculate bonus by department
- IT = 15%
- Finance = 12%
- HR = 10%
- Others = 5%
- Update bonus column
===== */

CREATE OR REPLACE PROCEDURE calculate_employee_bonus
(
    p_employee_id IN employees.employee_id%TYPE
)
AS
    v_salary      employees.salary%TYPE;
    v_department  employees.department%TYPE;
    v_bonus_rate  NUMBER;
    v_bonus       NUMBER;
BEGIN
    SELECT salary, department
    INTO v_salary, v_department
    FROM employees
    WHERE employee_id = p_employee_id
    AND status = 'ACTIVE';

    IF v_department = 'IT' THEN
        v_bonus_rate := 0.15;
    ELSIF v_department = 'Finance' THEN
        v_bonus_rate := 0.12;
    ELSIF v_department = 'HR' THEN
        v_bonus_rate := 0.10;
    ELSE
        v_bonus_rate := 0.05;
    END IF;

    v_bonus := v_salary * v_bonus_rate;

    UPDATE employees
    SET bonus = v_bonus
    WHERE employee_id = p_employee_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee bonus calculated successfully.');
```

```

    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || p_employee_id);
    DBMS_OUTPUT.PUT_LINE('Bonus Amount: ' || v_bonus);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Active employee not found.');
```

```

    WHEN OTHERS THEN

```

```

        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

```

/* Calling Employee Bonus Procedure */

```

```

SQL> BEGIN
      calculate_employee_bonus(1);
      calculate_employee_bonus(2);
      calculate_employee_bonus(3);
END;
/
  2      3      4      5      6 Employee bonus calculated successfully.
Employee ID: 1
Bonus Amount: 135000
Employee bonus calculated successfully.
Employee ID: 2
Bonus Amount: 60000
Employee bonus calculated successfully.
Employee ID: 3
Bonus Amount: 144000

PL/SQL procedure successfully completed.

```

```

/* =====
LAB 2: TELECOM RECHARGE PROCEDURE

Requirement:
- Recharge telecom account
- Validate subscriber
- Validate recharge amount
- Update balance
===== */

```

```

CREATE OR REPLACE PROCEDURE telecom_recharge
(
    p_subscriber_id    IN telecom_accounts.subscriber_id%TYPE,
    p_recharge_amount  IN NUMBER
)
AS
    v_status  telecom_accounts.status%TYPE;
    v_balance telecom_accounts.balance%TYPE;

    e_invalid_amount EXCEPTION;
    e_inactive_user   EXCEPTION;
BEGIN
    IF p_recharge_amount <= 0 THEN
        RAISE e_invalid_amount;
    END IF;

    SELECT status, balance
    INTO v_status, v_balance
    FROM telecom_accounts

```



```

WHERE subscriber_id = p_subscriber_id;

IF v_status <> 'ACTIVE' THEN
    RAISE e_inactive_user;
END IF;

UPDATE telecom_accounts
SET balance = balance + p_recharge_amount
WHERE subscriber_id = p_subscriber_id;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Recharge successful. ');
DBMS_OUTPUT.PUT_LINE('Subscriber ID: ' || p_subscriber_id);
DBMS_OUTPUT.PUT_LINE('Old Balance: ' || v_balance);
DBMS_OUTPUT.PUT_LINE('Recharge Amount: ' || p_recharge_amount);
DBMS_OUTPUT.PUT_LINE('New Balance: ' || (v_balance +
p_recharge_amount));

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Subscriber not found. ');
    WHEN e_invalid_amount THEN
        DBMS_OUTPUT.PUT_LINE('Recharge amount must be greater than
zero. ');
    WHEN e_inactive_user THEN
        DBMS_OUTPUT.PUT_LINE('Subscriber account is not active. ');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Recharge Error: ' || SQLERRM);
END;
/

/* Calling Telecom Recharge Procedure */

```

```

SQL> BEGIN
    telecom_recharge(1, 10000);
    telecom_recharge(20, 5000);
END;
/
  2      3      4      5 Recharge successful.
Subscriber ID: 1
Old Balance: 5000
Recharge Amount: 10000
New Balance: 15000
Subscriber not found.

PL/SQL procedure successfully completed.

```

```

/* =====
LAB 3: BANK TRANSFER PROCEDURE

Requirement:
- Transfer money from one account to another account
- Validate sender account
- Validate receiver account
- Validate transfer amount
- Check sender balance
- Use transaction control
===== */

CREATE OR REPLACE PROCEDURE bank_transfer
(
    p_from_account_id IN bank_accounts.account_id%TYPE,
    p_to_account_id   IN bank_accounts.account_id%TYPE,
    p_amount          IN NUMBER
)
AS
    v_from_balance bank_accounts.balance%TYPE;
    v_from_status  bank_accounts.status%TYPE;
    v_to_status    bank_accounts.status%TYPE;

    e_invalid_amount      EXCEPTION;
    e_same_account        EXCEPTION;
    e_insufficient_balance EXCEPTION;
    e_inactive_account    EXCEPTION;
BEGIN
    IF p_amount <= 0 THEN
        RAISE e_invalid_amount;
    END IF;

    IF p_from_account_id = p_to_account_id THEN
        RAISE e_same_account;
    END IF;

    SELECT balance, status
    INTO v_from_balance, v_from_status
    FROM bank_accounts
    WHERE account_id = p_from_account_id;

    SELECT status
    INTO v_to_status
    FROM bank_accounts
    WHERE account_id = p_to_account_id;

    IF v_from_status <> 'ACTIVE' OR v_to_status <> 'ACTIVE' THEN
        RAISE e_inactive_account;
    END IF;

    IF p_amount > v_from_balance THEN
        RAISE e_insufficient_balance;
    END IF;

```

```

UPDATE bank_accounts
SET balance = balance - p_amount
WHERE account_id = p_from_account_id;

UPDATE bank_accounts
SET balance = balance + p_amount
WHERE account_id = p_to_account_id;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Bank transfer successful.');
```

```

DBMS_OUTPUT.PUT_LINE('From Account: ' || p_from_account_id);
DBMS_OUTPUT.PUT_LINE('To Account: ' || p_to_account_id);
DBMS_OUTPUT.PUT_LINE('Transfer Amount: ' || p_amount);

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Transfer Error: Account not found.');
```

```

  WHEN e_invalid_amount THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Transfer Error: Amount must be greater than
zero.');
```

```

  WHEN e_same_account THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Transfer Error: Sender and receiver cannot
be same account.');
```

```

  WHEN e_inactive_account THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Transfer Error: One of the accounts is not
active.');
```

```

  WHEN e_insufficient_balance THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Transfer Error: Insufficient balance.');
```

```

  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Unexpected Transfer Error: ' || SQLERRM);
END;
/

/* Calling Bank Transfer Procedure */
```

```

SQL> BEGIN
      bank_transfer(1, 1, 2000000);
END;
/
  2      3      4 Transfer Error: Sender and receiver cannot be same account.

PL/SQL procedure successfully completed.
```

```

/* =====
FINAL CHECKING
===== */

```

```
SQL> SELECT * FROM employees ORDER BY employee_id;
```

```
EMPLOYEE_ID
```

```
EMPLOYEE_NAME
```

```
DEPARTMENT
```

```
SALARY
```

```
BONUS
```

```
STATUS
```

```
1
```

```
Aung Aung
```

```
IT
```

```
900000
```

```
135000
```

```
ACTIVE
```

```
EMPLOYEE_ID
```

```
EMPLOYEE_NAME
```

```
DEPARTMENT
```

```
SALARY
```

```
BONUS
```

```
STATUS
```

```
2
```

```
Mg Mg
```

```
HR
```

```
600000
```

```
60000
```

```
ACTIVE
```

```
EMPLOYEE_ID
```

```
EMPLOYEE_NAME
```

```
DEPARTMENT
```

```
SALARY
```

```
BONUS
```

```
STATUS
```

```
3
```

```
Su Su
```

```
Finance
```

```
1200000
```

```
144000
```

```
ACTIVE
```

```
SQL> SELECT * FROM telecom_accounts ORDER BY subscriber_id;
```

```
SUBSCRIBER_ID MSISDN
```

```
CUSTOMER_NAME
```

```
BALANCE STATUS
```

```
1 0911111111  
Aung Aung  
15000 ACTIVE
```

```
2 0922222222  
Mg Mg  
1000 ACTIVE
```

```
SQL> SELECT * FROM bank_accounts ORDER BY account_id;
```

```
ACCOUNT_ID
```

```
ACCOUNT_NAME
```

```
BALANCE STATUS
```

```
1  
Aung Aung  
1000000 ACTIVE
```

```
2  
Mg Mg  
500000 ACTIVE
```